Федеральное государственное автономное образовательное учреждение высшего образования "Национальный исследовательский университет "Высшая школа экономики" Московский институт электроники и математики им. А. Н. Тихонова НИУ ВШЭ Департамент прикладной математики

Практическая работа №4

«Создание и отслеживание процессов в операционной системе Linux»

По дисциплине «Операционные системы»

Группа: БПМ244

Студент: Артемьев Никита

Москва 2026

Цель:

Изучить создание и отслеживание процессов в операционной системе Linux с помощью программ на языке C.

Process и Parent Process ID

Значение PID: PID (Process ID) — это уникальный идентификатор процесса в операционной системе. Каждый раз, когда вы запускаете программу, создается новый процесс, которому присваивается новый PID. Поэтому при каждом запуске программы значение PID будет разным.

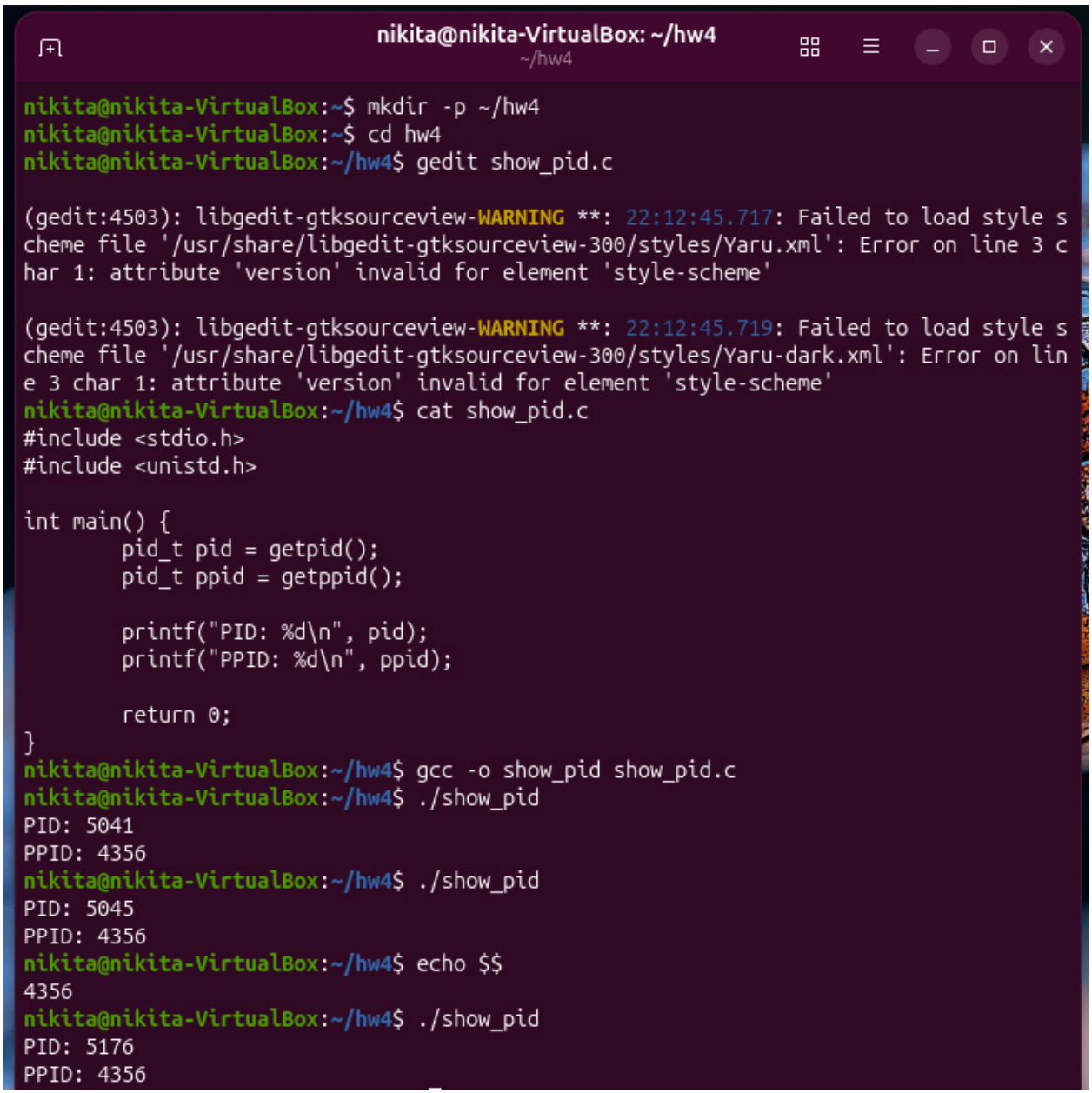
Значение PPID: PPID (Parent Process ID) — это идентификатор родительского процесса, который создал текущий процесс. Обычно, когда вы запускаете программу из командной строки, родительским процессом будет оболочка (например, bash или zsh), и её PID будет значением PPID для вашей программы.

Необычные значения PPID: Если родительский процесс завершится раньше дочернего, операционная система назначит дочернему процессу новым родителем процесс init (или systemd в современных Linux-системах). В этом случае PPID может измениться на PID процесса init (PID 1). Это может произойти, если вы запускаете программу в фоне и затем закрываете терминал или используете команду nohup.

Необычный характер печати: В нормальных условиях вывод будет последовательным и предсказуемым. Однако, если программа будет выполняться в специфических условиях (например, в многозадачной среде или при использовании параллельного выполнения), может наблюдаться несинхронизированный вывод или перемешивание строк (если несколько процессов пишут в один и тот же файл или терминал).

1.Вывод PID и PPID текущего процесса

В домашнем каталоге создан каталог hw4. В файле show_pid.c написана программа, которая выводит идентификатор текущего процесса и идентификатор родительского процесса. Для этого используются системные вызовы getpid() и getppid(). Программа скомпилирована и запущена несколько раз подряд:



При каждом запуске ядро создаёт новый процесс и выдаёт ему новый PID, поэтому номер меняется: 5041, затем 5045. Шаг не равен 1, потому что между запусками в системе успели появиться другие процессы, и промежуточные номера достались им. Для этого задания важно, что каждый запуск получает свой PID.

PPID три раза равен 4356. Это PID командной оболочки bash в данном терминале: оба раза программу запустил один и тот же процесс, поэтому номер родителя не меняется. Команда echo $$ выводит PID оболочки и подтверждает, что он совпадает с PPID программы.

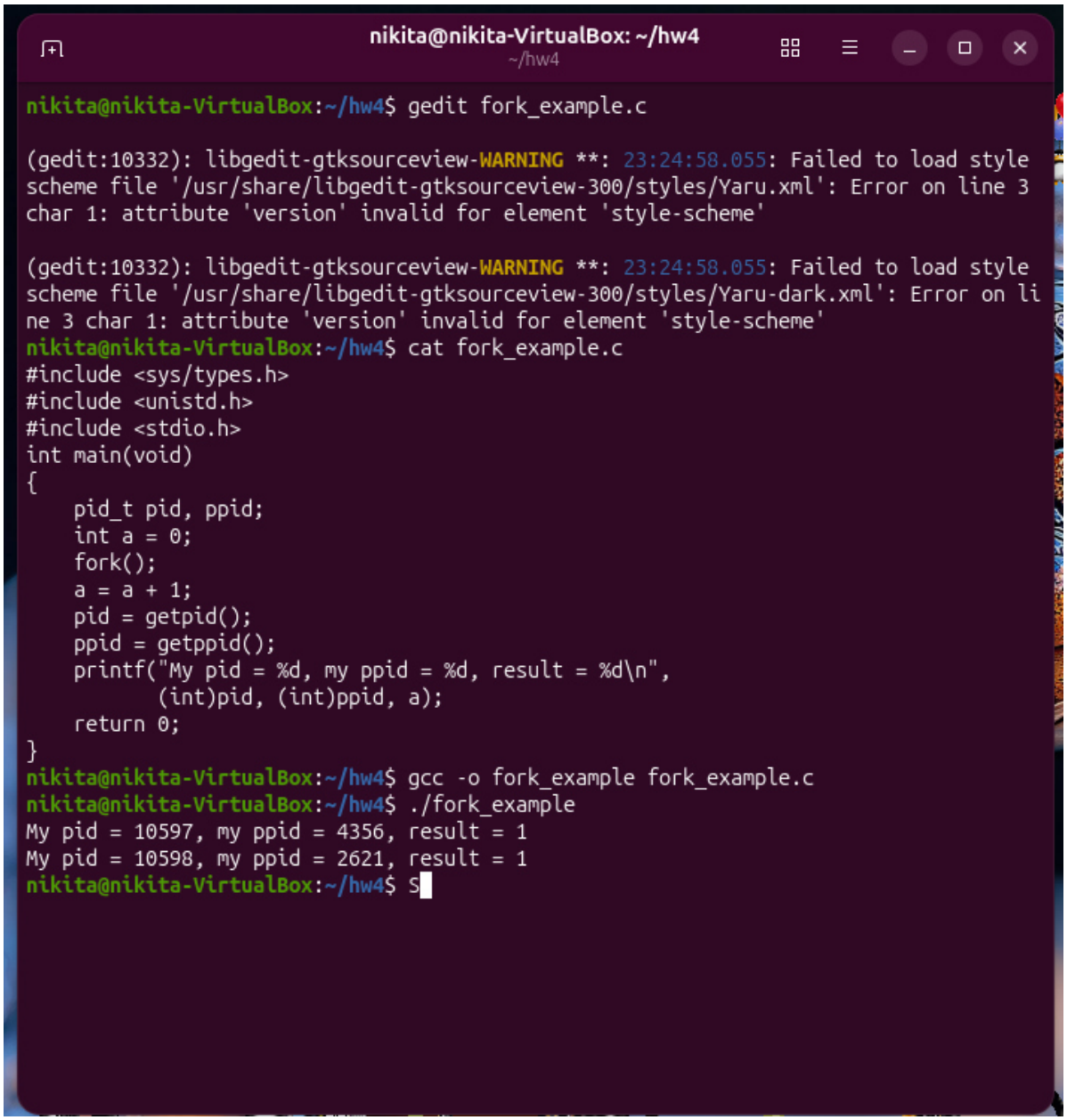
PPID был бы необычным, если бы родитель завершился раньше потомка. Тогда ядро назначает потомку нового родителя — systemd (в современных системах) или init, и PPID становится равным 1. Так происходит, если процесс оставили в фоне и закрыли терминал. В этом опыте терминал оставался открытым, родитель не завершался, поэтому PPID не изменился.

Печать обычная: каждый процесс полностью вывел две строки и завершился, затем запустился следующий. Строки не перемешались, потому что в терминал в каждый момент писал только один процесс. Перемешанный вывод появляется, когда несколько процессов печатают одновременно; это будет видно после fork().

2.Системный вызов fork(), вторая программа и ее модернизация

Новый процесс в Linux создаётся вызовом fork(). Родитель и ребёнок продолжают выполнение с места после fork(), но получают разные возвращаемые значения: в родителе — PID ребёнка, в ребёнке — 0.

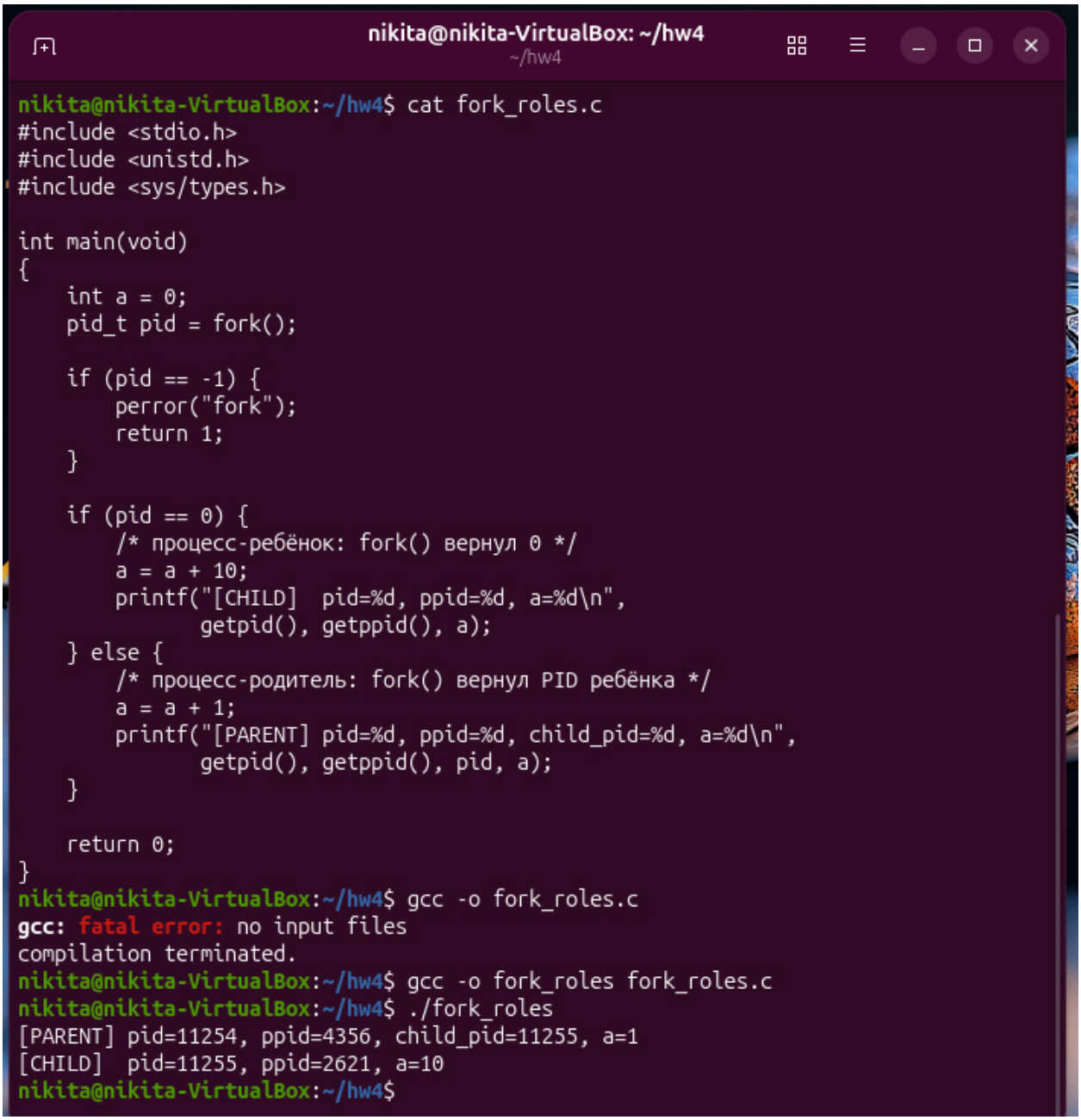
пример из методички



Программа без проверки возврата fork() увеличивает a в обоих процессах; в выводе у обеих строк result = 1.

Задание: разное поведение

Программа fork_roles.c сохраняет результат fork() в переменную pid и по нему выбирает ветку: ребёнок увеличивает a на 10 и печатает одну строку, родитель — на 1 и печатает другую, включая child_pid.



После fork() работают два процесса. Ребёнок определяется условием pid == 0; в родителе pid равен идентификатору созданного процесса. PPID ребёнка совпадает с PID родительского процесса программы — видна иерархия. Переменная a у каждого процесса своя копия, поэтому в родителе и ребёнке разные значения. Порядок строк в терминале может меняться от запуска к запуску — это следствие параллельного выполнения и асинхронного вывода в stdout.

3. exec — ребёнок запускает другую программу

На базе программы с fork() дочерний процесс вместо своего кода вызывает execl() и запускает системную утилиту ls с ключом -l. Родительский процесс ожидает завершения ребёнка системным вызовом waitpid().

```
nikita@nikita-VirtualBox:~/hw4$ cat fork_exec.c
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main(void)
{
    pid_t pid = fork();

    if (pid == -1) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {
        /* Ребёнок: запускаем другую программу (например, ls) */
        execl("/bin/ls", "ls", "-l", (char *)NULL);

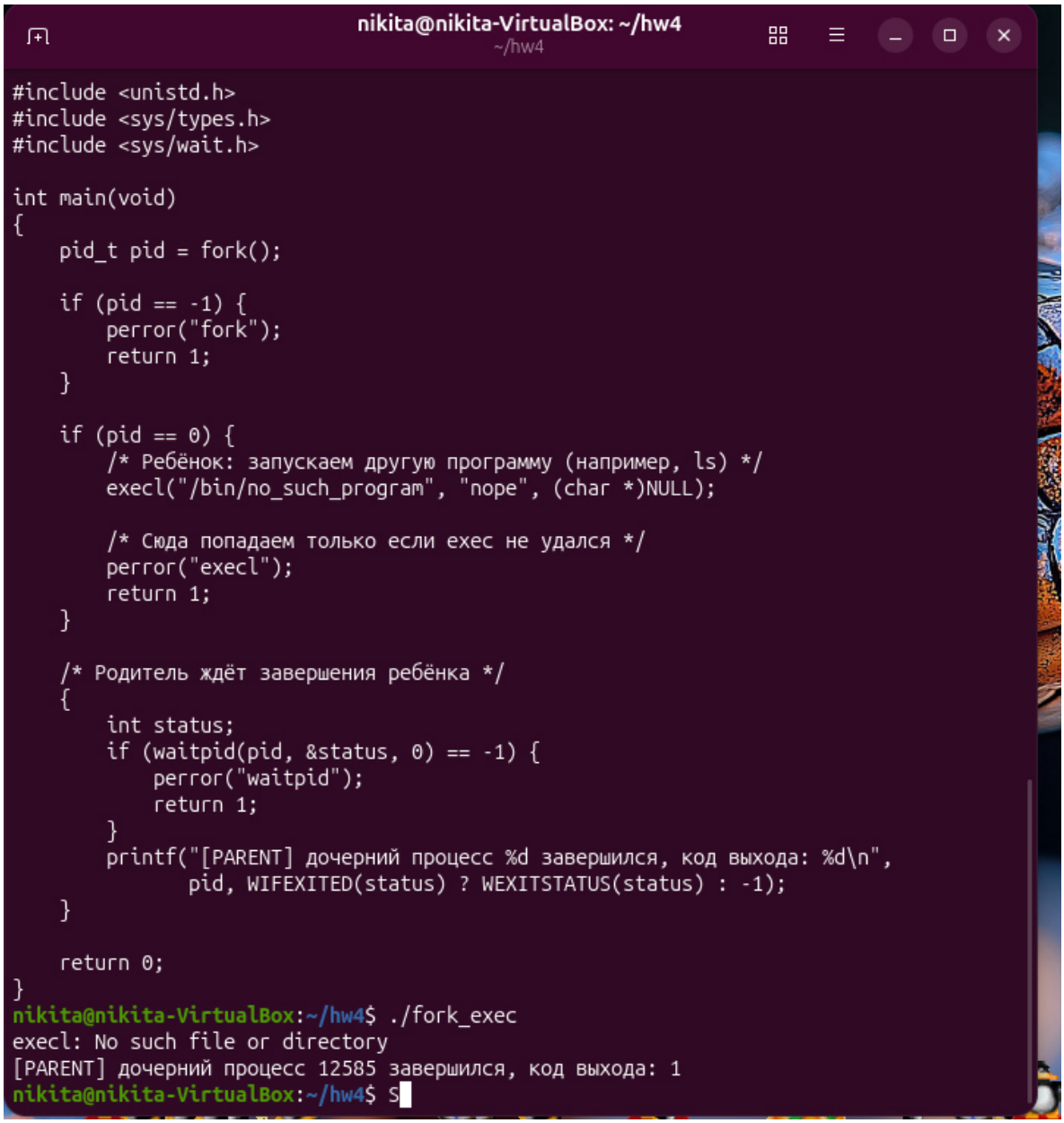
        /* Сюда попадаем только если exec не удался */
        perror("execl");
        return 1;
    }

    /* Родитель ждёт завершения ребёнка */
    {
        int status;
        if (waitpid(pid, &status, 0) == -1) {
            perror("waitpid");
            return 1;
        }
        printf("[PARENT] дочерний процесс %d завершился, код выхода: %d\n",
               pid, WIFEXITED(status) ? WEXITSTATUS(status) : -1);
    }

    return 0;
}
nikita@nikita-VirtualBox:~/hw4$
```

```
nikita@nikita-VirtualBox:~/hw4$ gcc -o fork_exec fork_exec.c
nikita@nikita-VirtualBox:~/hw4$ ./fork_exec
total 80
-rwxrwxr-x 1 nikita nikita 16088 Oct  9 23:25 fork_example
-rw-rw-r-- 1 nikita nikita   291 Oct  9 23:25 fork_example.c
-rwxrwxr-x 1 nikita nikita 16176 Oct  9 23:36 fork_exec
-rw-rw-r-- 1 nikita nikita   916 Oct  9 23:35 fork_exec.c
-rwxrwxr-x 1 nikita nikita 16128 Oct  9 23:29 fork_roles
-rw-rw-r-- 1 nikita nikita   639 Oct  9 23:27 fork_roles.c
-rwxrwxr-x 1 nikita nikita 16048 Oct  9 22:17 show_pid
-rw-rw-r-- 1 nikita nikita   175 Oct  9 22:15 show_pid.c
[PARENT] дочерний процесс 12088 завершился, код выхода: 0
```

Если указать несуществующий исполняемый файл, execl возвращает управление, вызывается perror("execl"), выводится сообщение об ошибке.



После успешного exec образ процесса заменяется новой программой, поэтому код после execl не выполняется. Строки с perror и return нужны только при сбое. Родитель с waitpid не завершается раньше ребёнка и получает код завершения дочернего процесса.

Вывод

В ходе практической работы изучены идентификаторы процессов в Linux и базовые механизмы их создания.

С помощью программы show_pid получены PID и PPID текущего процесса. При повторных запусках PID каждый раз новый, так как ядро создаёт отдельный процесс. PPID при запуске из одного терминала совпадает с PID оболочки и остаётся неизменным, пока родитель жив. Наблюдалось, что при завершении родителя раньше ребёнка PPID ребёнка может измениться — процесс усыновляется другим родителем в системе.

Системный вызов fork() создаёт копию процесса: один вызов, два процесса с разными возвращаемыми значениями (0 в ребёнке, PID ребёнка в родителе). В программе fork_roles родитель и ребёнок выполняют разный код и выводят разные значения локальной переменной, что подтверждает раздельные адресные пространства после fork().

В программе fork_exec дочерний процесс с помощью execl() заменяет свой образ на внешнюю программу (ls -l). После успешного exec управление в исходный код не возвращается. Обработка ошибки через perror показана на несуществующем пути к исполняемому файлу. Родитель с waitpid() дожидается завершения ребёнка и получает код выхода.

Таким образом, освоены связка getpid/getppid, создание процессов через fork() и запуск другой программы в потомке через семейство exec, а также ожидание завершения дочернего процесса.